MODUL 6 · TYPESCRIPT DALAM NEXT.JS

# tsconfig.json: Aturan Main Compiler TypeScript

Memahami setiap opsi bawaan create-next-app

# Apa itu tsconfig.json?

File konfigurasi di root project yang memberi tahu TypeScript **bagaimana kode kita harus dicek.**

1. **Dibaca tiga sistem**
   Editor (garis merah & autocomplete), perintah tsc, dan proses next build

2. **Dibuat otomatis oleh Next.js**
   Dari create-next-app, atau saat kita mengganti file menjadi .ts/.tsx lalu menjalankan next dev

3. **Punya tiga bagian utama**
   compilerOptions (aturan pengecekan), include dan exclude (file mana yang dicek)

# tsconfig.json bawaan create-next-app

tsconfig.json (1/2)

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
```

tsconfig.json (2/2)

```
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./*"] }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

# Opsi Pengecekan pada tsconfig

1. **strict: true**
   Paket pengecekan ketat (noImplicitAny, strictNullChecks, dan lainnya) — inilah yang membuat tipe benar-benar berguna

2. **skipLibCheck: true**
   Lewati pengecekan file deklarasi (.d.ts) di node_modules — lebih cepat; kode kita sendiri tetap dicek

3. **noEmit: true**
   TypeScript hanya mengecek, tidak menghasilkan file JavaScript — kompilasinya dikerjakan Next.js

4. **isolatedModules: true**
   Memastikan tiap file aman diproses sendiri-sendiri oleh transpiler

# Efek strict

Dua kesalahan ini hanya tertangkap saat strict aktif. Dengan strict mati, keduanya lolos tanpa peringatan.

contoh-strict.ts

```
function sapa(nama) {
  return nama;
}
// Error: Parameter 'nama' implicitly has an 'any' type.

const el: string | null = null;
el.length;
// Error: 'el' is possibly 'null'.
```

# Opsi Bahasa & Modul pada tsconfig

1. **target: ES2017**
   Versi JavaScript yang menjadi acuan fitur bahasa

2. **lib: dom, dom.iterable, esnext**
   API yang dikenali mulai document, window, dan fitur JavaScript terbaru

3. **module: esnext + moduleResolution: bundler**
   Import dicari seperti bundler, boleh tanpa ekstensi file

4. **esModuleInterop, resolveJsonModule, allowJs**
   Kompatibilitas import, import file .json, dan file .js boleh ada

5. **jsx: react-jsx**
   JSX modern — tidak perlu import React di setiap file

# Efek Opsi Bahasa & Modul

Tiga baris import dan akses berikut bisa dipakai karena opsi di slide sebelumnya.

contoh-modul.ts

```
import { hitung } from "./utils";
// tanpa ekstensi .ts → OK (moduleResolution: bundler)

import settings from "./settings.json";
// import file JSON → OK (resolveJsonModule)

const judul = document.title;
// akses document → OK (lib: dom)
```

# Opsi Khusus Next.js

1. **plugins: next**
   Plugin Next.js untuk editor: mengecek 'use client' dan pemakaian hooks

2. **incremental: true**
   Menyimpan hasil pengecekan sebelumnya agar type check berikutnya lebih cepat

3. **paths: { "@/*": ["./*"] }**
   Alias import: @/ menunjuk ke root project

4. **include & exclude**
   next-env.d.ts dan .next/types berisi tipe yang digenerate Next.js (misalnya PageProps), node_modules dikecualikan

# Alias Import dengan paths

Satu baris paths menghilangkan rantai ../../../ yang sulit dibaca.

app/dashboard/page.tsx

```
// Tanpa alias — rawan keliru menghitung jumlah ../
import { Button } from "../../components/button";

// Dengan alias "@/*": ["./*"] — path selalu dari root
import { Button } from "@/components/button";
```

# Kesalahan Umum Soal tsconfig.json

### Mematikan strict karena error terlalu banyak

Error-nya tidak hilang, hanya disembunyikan. Bug-nya tetap ada di kode. Perbaiki tipenya, jangan matikan pengecekannya.

### Mengedit next-env.d.ts atau mengaktifkan ignoreBuildErrors

next-env.d.ts digenerate Next.js dan akan tertimpa (masukkan ke .gitignore). ignoreBuildErrors: true membuat build lolos meski ada error tipe. Ini berbahaya.

### Editor belum berubah setelah tsconfig.json diedit

Jalankan "TypeScript: Restart TS Server" dari command palette. Untuk CI, cek error tipe dengan tsc --noEmit sebelum deploy.